

Product Microservice

REST API Design, Implementation and Testing

Endpoint Definition

The product service exposes five RESTful endpoints grouped under the /products resource. Each endpoint maps to a distinct CRUD operation aligned to the Product Management business capability. All endpoints consume and produce application/json payloads and are served over HTTPS.

Method	Endpoint	Description	Expected Status
GET	/products	Retrieve a list of all products in the catalogue	200 OK
GET	/products/{id}	Retrieve a single product by its unique identifier	200 OK
POST	/products	Create a new product record in the catalogue	201 Created
PUT	/products/{id}	Update one or more attributes of an existing product	200 OK
DELETE	/products/{id}	Remove a product record permanently from the catalogue	204 No Content

Standard Product Payload Schema

All endpoints that return a product resource use the following JSON structure:

```
{
  "id": 101,
  "name": "Wireless Headphones",
  "price": 129.99,
  "category": "Electronics",
  "availability": true,
  "region": "US-East"
}
```

Required fields for POST: id, name, price, category, availability, region. All fields are accepted on PUT (partial updates supported).

HTTP Status Code Reference

Status Code	Type	Meaning in this service
200 OK	Success	GET and PUT operations completed successfully — response body contains the resource
201 Created	Success	POST operation completed — new product record created, full resource returned in body
204 No Content	Success	DELETE operation completed — product removed, no response body returned
400 Bad Request	Client Error	Request payload is missing required fields, contains invalid types, or violates constraints
404 Not Found	Client Error	No product with the specified id exists in the catalogue
500 Internal Error	Server Error	Unexpected runtime error on the service side — client should retry with exponential back-off

Sample Requests and Responses

The following five scenarios were validated using Swagger UI (served automatically by FastAPI at /docs) and verified with Postman. Each scenario documents the HTTP method, target endpoint URL, request payload where applicable, the full response payload, and the returned HTTP status code.

Test 1 — Successful Product Creation (POST)

Method	POST
Endpoint	http://localhost:8000/products
Request Body	<pre>{ "id": 101, "name": "Wireless Headphones", "price": 129.99, "category": "Electronics", "availability": true, "region": "US-East" }</pre>
Response	<pre>{ "id": 101, "name": "Wireless Headphones", "price": 129.99, "category": "Electronics", "availability": true, "region": "US-East" }</pre>
HTTP Status	201 Created

Note: The service echoes back the full created resource. Status 201 confirms a new record was written to the catalogue.

Test 2 — Successful Retrieval of All Products (GET)

Method	GET
Endpoint	http://localhost:8000/products
Request Body	(none)
Response	<pre>{ "products": [{ "id": 101, "name": "Wireless Headphones", "price": 129.99, "category": "Electronics", "availability": true, "region": "US-East" }], "count": 1 }</pre>
HTTP Status	200 OK

Note: No request body is required. The response wraps the product array in a top-level products key and includes a count field to support pagination planning.

Test 3 — Successful Retrieval of Single Product (GET by ID)

Method	GET
Endpoint	http://localhost:8000/products/101
Request Body	(none)
Response	<pre>{ "id": 101, "name": "Wireless Headphones", "price": 129.99, "category": "Electronics", "availability": true, "region": "US-East" }</pre>
HTTP Status	200 OK

Note: The product id is passed as a path parameter. The service returns the matching resource directly as the root object.

Test 4 — Successful Product Update (PUT)

Method	PUT
Endpoint	http://localhost:8000/products/101
Request Body	<pre>{ "price": 109.99, "availability": false }</pre>
Response	<pre>{ "id": 101, "name": "Wireless Headphones", "price": 109.99, "category": "Electronics", "availability": false, "region": "US-East" }</pre>
HTTP Status	200 OK

Note: Only the fields included in the request body are updated (partial update). The service merges the patch and returns the full updated resource.

Test 5 — Successful Product Deletion (DELETE)

Method	DELETE
Endpoint	http://localhost:8000/products/101
Request Body	(none)
Response	(empty body — no content returned on successful deletion)
HTTP Status	204 No Content

Note: Status 204 confirms the resource was removed. No response body is returned, consistent with REST conventions for DELETE.

Test 6 — Error Scenario: Missing Required Fields (POST)

Method	POST
Endpoint	http://localhost:8000/products
Request Body	<pre>{ "name": "Smart Watch" }</pre>
Response	<pre>{ "detail": [{ "loc": ["body", "id"], "msg": "field required", "type": "value error.missing" }] }</pre>

	<pre> }, { "loc": ["body", "price"], "msg": "field required", "type": "value_error.missing" }] }</pre>
HTTP Status	400 Bad Request

Note: The service validates each required field (id, price, category, availability, region) before persisting. Missing fields are enumerated in the detail array, enabling the client to correct all errors in a single retry.

Test 7 — Error Scenario: Product Not Found (GET)

Method	GET
Endpoint	http://localhost:8000/products/999
Request Body	(none)
Response	<pre>{ "detail": "Product with id 999 not found" }</pre>
HTTP Status	404 Not Found

Note: When no record matches the requested id, the service returns 404 with a human-readable detail message. This prevents the client from misinterpreting an empty response as a valid empty result.

Architectural Alignment

The product service is designed as a self-contained microservice that encapsulates a single, well-defined business capability: the management of product catalogue data. By exposing only a clean RESTful contract over HTTPS, it decouples consumers — including the order service, inventory service, and third-party marketplace partners — from any internal implementation detail, satisfying the loose coupling requirement identified in the stakeholder register.

Because the service owns its own in-memory (and, in production, its own dedicated database) store, it can be deployed, scaled, and updated independently of every other service in the platform. During the Black Friday peak identified by the VP of E-Commerce, the product service can be horizontally scaled in isolation — adding compute capacity only where it is needed — without requiring a full-stack redeployment or any downtime for unrelated services.

Within a hybrid cloud architecture, this service boundary also supports the compliance requirements raised by the Security and Compliance Director. Regional deployments of the product service can be

constrained to specific cloud regions or on-premises nodes, ensuring that product and pricing data for EU customers, for example, is processed and stored within GDPR-governed infrastructure, while APAC deployments remain within PDPA-compliant boundaries — all without changing the shared service contract.

Implementation Notes

Technology Stack

Component	Detail
Language	Python 3.12
Framework	FastAPI 0.115 — chosen for automatic OpenAPI / Swagger UI generation
Validation	Pydantic v2 models — enforces types and constraints on all payloads
Storage	In-memory Python dict — no external database required for this lab
API Testing Tool	Swagger UI (auto-generated at /docs) and Postman collection
Serialization	JSON — application/json content type on all endpoints

Service Entry Point

```
# Run the service locally
uvicorn main:app --host 0.0.0.0 --port 8000 --reload

# Swagger UI available at:
http://localhost:8000/docs

# OpenAPI JSON schema at:
http://localhost:8000/openapi.json
```

The service can be containerised with a standard Dockerfile and deployed to any Kubernetes cluster or cloud-managed container runtime. The same REST contract is preserved regardless of the deployment target, supporting the hybrid cloud deployment model.